



Scheduling

# Task Scheduling

# Task Scheduling

Task scheduling in Spring Boot refers to the process of executing tasks or methods at specified times or intervals, either once or repeatedly, without manual intervention. Spring Boot provides built-in support for scheduling using the **@Scheduled** annotation and **TaskScheduler**.

Task Use cases:

- Logging user activity in a separate thread.
- Sending emails or SMS notifications asynchronously after a user action.
- Processing multiple image uploads, database queries, or API calls in parallel.
- Cleaning up expired records in the database.
- Archiving logs.
- Fetching data from external APIs regularly.
- Performing regular health checks on external systems or services.

# @Scheduled Annotation

## How It Works:

1. Enable Scheduling: To use @Scheduled, you must first enable scheduling in your application by adding the @EnableScheduling annotation to a configuration class.
2. Annotate Methods: Add @Scheduled to any method you want to schedule. The method must:
  - Be void.
  - Take no arguments.

# Key parameters of @Scheduled

1. **fixedRate**: Defines the interval between the start of consecutive executions, in milliseconds. Ignores the duration of the task itself.
2. **fixedDelay**: Defines the delay between the end of one execution and the start of the next, in milliseconds.
3. **initialDelay**: Adds an initial delay (in milliseconds) before the first execution of a task. Can be used with fixedRate or fixedDelay.
4. **cron**: Allows you to specify a more complex schedule using a cron expression.

```
@Scheduled(initialDelay = 3000, fixedRate = 5000)
public void runWithInitialDelay() {
    System.out.println("Initial Delay: " + LocalDateTime.now());
}
```

# How @Scheduled works?

When you annotate a method with @Scheduled, Spring:

1. Registers the method as a scheduled task.
2. Delegates task execution to a TaskScheduler implementation.
3. Uses a thread pool to run tasks concurrently (by default, ScheduledExecutorService under the hood).

By default, @Scheduled uses a single-threaded executor, which means all tasks run sequentially. For concurrent execution, define a custom TaskScheduler with a thread pool (ThreadPoolTaskScheduler).

@Scheduled → Uses Spring's TaskScheduler.

TaskScheduler → Uses ThreadPoolTaskScheduler (Spring's implementation).

ThreadPoolTaskScheduler → Internally delegates task execution to ScheduledExecutorService.

# TaskScheduler

**TaskScheduler** is an interface in Spring's scheduling framework, used for scheduling tasks to be executed at a specific time or periodically.

- **Dynamic Scheduling:** You can schedule tasks programmatically using different schedules (fixed rate, fixed delay, cron expressions, etc.).
- **Fine-grained Control:** You can create, cancel, or reschedule tasks dynamically.
- **Supports Cron Expressions:** You can define tasks with cron-based schedules.
- **Thread Pooling:** Often used with `ThreadPoolTaskScheduler` for efficient thread management.

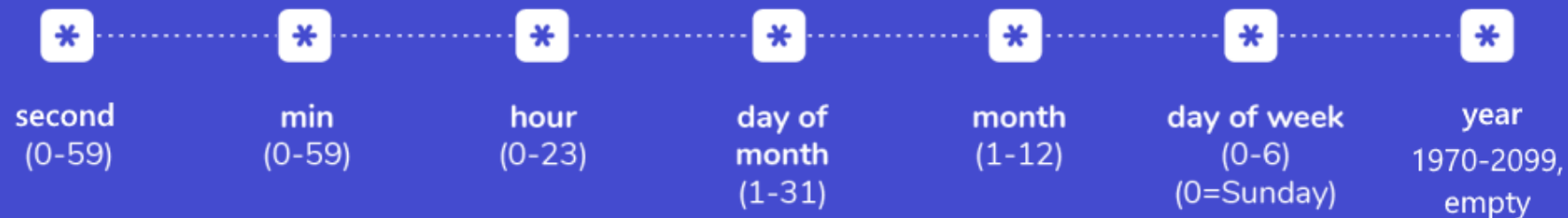
# Configuring TaskScheduler

Spring Boot provides a default implementation of TaskScheduler via the ThreadPoolTaskScheduler, which is built on top of Java's ScheduledExecutorService.

```
@Configuration
public class SchedulerConfig {

    @Bean
    public TaskScheduler taskScheduler() {
        ThreadPoolTaskScheduler scheduler = new ThreadPoolTaskScheduler();
        scheduler.setPoolSize(5); // Number of threads
        scheduler.setThreadNamePrefix("TaskSchedulerThread-");
        scheduler.initialize();
        return scheduler;
    }
}
```

## CRON EXPRESSIONS QUICK REFERENCE GUIDE



- range of values ..... MON-FRI,1-5
- \* any value in range ..... \*
- / step values ..... \*/10
- ' value list separator ..... 0,20,40

```
@Scheduled(cron = "0 0/1 * * * ?") // Runs every minute
public void runEveryMinute() {
    System.out.println("Task executed at: " + LocalDateTime.now());
}
```



s m h D M day

| Cron Expression                   | Explanation                                              |
|-----------------------------------|----------------------------------------------------------|
| <code>0 * * * * *</code>          | Every minute at the 0th second                           |
| <code>0 0 12 * * *</code>         | Every day at 12:00 PM (noon)                             |
| <code>0 0 0 1 * *</code>          | At midnight on the first day of every month              |
| <code>0 0 12 1 * 1</code>         | Every first Monday of every month at 12:00 PM            |
| <code>0 0 0 1 1 *</code>          | At midnight on January 1st                               |
| <code>0 0 9-17 * * MON-FRI</code> | Every weekday (Monday to Friday) from 9:00 AM to 5:00 PM |
| <code>0 0/10 * * * *</code>       | Every 10 minutes                                         |
| <code>0 30 8-18/2 * * *</code>    | Every 2 hours between 8 AM and 6 PM, starting at 8:30 AM |
| <code>0 0 0 * * 6</code>          | Every Saturday at midnight                               |
| <code>0 15 10 * * ?</code>        | Every day at 10:15 AM                                    |

s m h D M day

| Cron Expression                      | Explanation                                                             |
|--------------------------------------|-------------------------------------------------------------------------|
| <code>0 */5 8-20 * * MON-FRI</code>  | Every 5 minutes, between 8 AM and 8 PM, on weekdays (Monday to Friday). |
| <code>0 0 0 L * ?</code>             | At midnight on the last day of every month.                             |
| <code>0 0 12 1/3 * ?</code>          | At noon on the 1st day of every third month.                            |
| <code>0 15 10 ? * 1#2</code>         | At 10:15 AM on the second Monday of every month.                        |
| <code>0 0 1 1 * ?</code>             | At midnight on January 1st, every year (New Year's Day).                |
| <code>0 0/10 8-18 * * MON-FRI</code> | Every 10 minutes between 8 AM and 6 PM, Monday through Friday.          |
| <code>0 0 0 25 12 ?</code>           | At midnight on December 25th, every year (Christmas Day).               |
| <code>0 0 10,14,16 * * ?</code>      | At 10 AM, 2 PM, and 4 PM every day.                                     |
| <code>0 30 7-18/2 * * ?</code>       | At 30 minutes past every second hour between 7 AM and 6 PM.             |
| <code>0 0 0 1 1/2 ?</code>           | At midnight on January 1st and every 2nd month after that.              |

